BUILD A LARGE LANGUAGE MODEL

Capítulo 4 - Implementing a GPT model from scratch to generate text

Anotações:

- GPT possui uma arquitetura de redes neurais profundas de larga escala, feita para gerar novo texto uma palavra (token) por vez.
- **Parâmetros:** Pesos treináveis do modelo.
- **Pesos:** Variáveis internas do modelo que são ajustadas e otimizadas durante o processo de treinamento para minimizar uma função específica de perda (loss). Isso permite ao modelo aprender com os dados de treinamento.
- **GPT model:** Consiste em 1 ou mais blocos transformers contendo o módulo mascarado de multi-head attention.
- Camada de rede neural com tensor de 2.048 x 2.048 dimensional de pesos, cada elemento da matriz é um parâmetro, ou seja, 2.048 * 2.048 = 4.194.304 parâmetros na camada.
- **Dummy-GPTModel:** Arquitetura de placeholder para GPT. Consiste em token, embeddings posicionais, dropout, transformers blocos, camada final de normalização e uma camada de saída linear.
- O método forward descreve o fluxo dos dados através do modelo.
- **Fluxo dos dados entrando e saindo em um modelo GPT:** Geralmente a dimensão do token embedded de entrada é igual a dimensão de saída.
- **Tokenizador GPT2:** Uma palavra pode virar um único token ou ser dividida em partes menores. Isso depende do vocabulário e das regras de tokenização.
- **Normalizar ativações com normalização de camada:** Treinar redes com muitas camadas pode gerar problemas como desaparecimento os gradientes explodirem o que leva para treinamento instáveis e faz ser mais difícil para a rede ajustar seus pesos e assim faz a rede ter dificuldades em aprender os padrões subjacentes nos dados. Implementar uma camada de normalização melhora a estabilidade e eficiência do treinamento de rede neural. Ajusta-se as ativações (saídas) da rede para ter variância entre 0 e 1. Normalização, em modelos GPT e arquiteturas transformers modernas, é geralmente aplicada antes e depois de multi-head attention e antes da camada de saída final.
- **Função de ativação:** Consiste em uma função não linear, ReLU, que garante que a camada de saída tenha apenas valores positivos ao limitar entradas negativas a 0.
- **Parâmetro dim:** o dim não escolhe diretamente os números que você quer calcular, ele escolhe a dimensão ao longo da qual os números serão agrupados para realizar a operação. dim=0 calcula verticalmente; dim=1 calcula horizontalmente. O resultado fica menor porque os valores usados em cada cálculo são agrupados em um único resultado.

- **Normalização em camadas:** Normaliza as características de cada exemplo individualmente, sem depender dos outros exemplos do lote. Layer Normalization trata cada frase independentemente das outras.
- **Normalização em lotes:** Normaliza os valores considerando vários exemplos de um lote (batch). Batch Normalization considera informações do conjunto de exemplos.
- GPT usa layer normalization porque funciona independentemente do tamanho do lote, tornando o modelo mais flexível durante o treinamento e o uso.
- **GELU (Gaussian error linear unit):** função de ativação que tem um papel crucial no submódulo na rede neural. Pode ser implementada de várias maneiras, na prática é comum implementar uma aproximação computacionalmente mais barata obtida por meio de ajuste de curvas. Diferente da ReLU que zera todos os valores negativos, a GELU permite que alguns valores negativos continuem passando, isso ajuda o modelo a aprender padrões mais complexos durante treinamento. Os valores negativos geralmente ficam próximos de zero, mas não são necessariamente zerados, ou seja, os neurônios que recebem entradas negativas ainda podem contribuir para o processo de aprendizado, porém com menos medida que as entradas positivas.
- GELU e SwiGLU (Swish-gated linear unit) são mais complexas e suaves funções de ativação, elas oferecem melhorias na performance para modelos deep learning, diferentemente da ReLU.
- **FeedForward:** Módulo de alimentação direta, muito importante para os blocos transformers do GPT-2. Pega a representação de cada token, expande a quantidade de números internamente, aplica uma transformação não linear e depois volta ao tamanho original. Isso ajuda o modelo a aprender representações mais complexas dos dados. Exemplo: 1° camada linear 768 para 3072 representações (mais variadas combinações), ativação GELU (transformação não linear, transforma os valores de maneira suave, permitindo que diferentes sinais influenciem o processamento de formas diferentes, permite que a rede aprenda relações mais complexas), 2° camada linear de 3072 para 768 representações (mantém dimensão compatível com as outras partes do bloco, como a conexão residual que soma as representações transformadas com a que entrou no bloco)
- **Conexões de curto atalho (shortcut connections):** Também chamadas de skip connections ou conexões residuais, são um mecanismo muito importante nas redes neurais profundas. Em vez de obrigar os dados a passar apenas pelas camadas de processamento, criamos um caminho alternativo que leva a entrada diretamente para uma etapa posterior. Em redes muito profundas, os gradientes podem ficar cada vez menores à medida que voltam pelas camadas. Isso é chamado de vanishing gradient ou desaparecimento do gradiente. No shortcut a saída processada é somada a entrada original, assim a saída incorpora tanto a transformação

realizada pelas camadas quanto a informação da entrada original. Isso ajuda a preservar informações e facilita o fluxo dos gradientes.

- **Transformer block:** Bloco de construção fundamental do GPT. Combina vários conceitos, multi-head attention, normalização de camada, dropout, feed forward camadas e GELU. Quando um bloco transformer processa uma sequência de entrada, cada elemento é representado por um vetor com tamanho fixo, todas as operações dentro desse bloco são designadas a transformar esses vetores de uma forma que preserve suas dimensões. A ideia é que o self-attention mecanismo no multi-head attention bloco identifique e analise as relações entre elementos na sequência de entrada, já o feed forward modifica os dados individualmente em cada posição para um entendimento mais profundo, processamento da entrada e aumenta a capacidade do modelo ao lidar com padrões complexos de dados. A saída é um vetor de contexto que encapsula informações de toda a sequência de entrada.
- **Layer Normalization:** É aplicada antes da atenção e da rede FeedForward para manter os valores dos dados mais estáveis durante o processamento. Depois de cada componente, o dropout desativa aleatoriamente alguns valores durante o treinamento, ajudando a evitar o overfitting, quando o modelo se adapta demais aos dados de treinamento.
- **Pre-LayerNorm:** Normaliza os dados antes de passarem pela atenção ou pela FeedForward. Ele facilita o fluxo dos gradientes durante o treinamento, tornando mais fácil treinar redes profundas, como o GPT-2
- **Post-LayerNorm:** Normaliza os dados depois de passarem pela atenção ou pela FeedForward. Pode dificultar a estabilidade do treinamento, especialmente em redes com muitas camadas.
- O GPT começa transformando o texto em tokens, que são convertidos em vetores chamados token embeddings. Depois, adiciona informações de posição (positional embeddings), para que o modelo saiba a ordem dos tokens. Em seguida, esses dados passam por 12 blocos Transformer, no GPT-2 de 124 milhões de parâmetros. Cada bloco possui atenção, uma rede FeedForward, normalização e dropout. A atenção relaciona as informações entre os tokens, enquanto a FeedForward as processa. Esse processo se repete nos blocos para que o modelo aprenda padrões e relações no texto.
- **Weight tying:** É uma técnica em que duas partes do modelo GPT compartilham os mesmos pesos: a camada que transforma tokens em vetores (tok_emb) e a camada que prevê o próximo token (out_head). Isso reduz a quantidade de parâmetros e o uso de memória. No exemplo, aparecem 163 milhões de parâmetros porque as duas camadas são contadas separadamente. Ao considerar o weight tying, o total cai para aproximadamente 124 milhões, como no GPT-2 original. Quando as camadas são separadas, cada uma pode aprender valores diferentes para uma função diferente. Com o weight tying, as duas camadas precisam compartilhar os mesmos pesos, o que reduz a liberdade de aprendizado. Separadas, elas

podem se especializar em suas tarefas, o que pode melhorar o treinamento e o desempenho em alguns casos.

- **Como um modelo GPT gera textos:** Dado uma entrada de contexto “Hello, I am” com cada iteração a entrada de contexto cresce, o que permite o modelo gerar coerente e contextualmente apropriado texto, conseguindo construir a frase “Hello, I am a modelo ready to help”.
- **Converter tensores de saída para texto:** Decodificar os tensores de saída, selecionar tokens baseado em distribuição de probabilidade e converter esses tokens em linguagem humana. O GPT recebe um texto de entrada e calcula pontuações (logits) para todos os tokens que poderiam vir a seguir. Depois, usa a função softmax para transformar essas pontuações em probabilidades. O modelo escolhe um token, obtém seu ID e o converte novamente em texto. Esse novo token é adicionado ao texto anterior e enviado novamente ao modelo. Esse processo se repete várias vezes, permitindo que o GPT gere frases e textos completos, um token por vez.
- Greedy decoding: é uma forma de o GPT escolher o próximo token, ele sempre seleciona aquele que tem a maior probabilidade naquele momento. Porém, o texto pode ficar repetitivo ou previsível, porque o modelo sempre escolhe a opção mais provável, sem explorar outras possibilidades.